프로그래밍 기초 with C++

학습목표

- C++ 언어로 프로그램을 작성할 수 있다.

개요

프로그래밍에 대한 이해

프로그래밍(Programming)이란 무엇일까? 프로그램(Program)이라는 단어는 본래 업무를 수행하기 위한 절차를 뜻하는 용어로 (컴퓨터) 프로그래밍이라는 것은 컴퓨터가 우리의 목적을 달성할 수 있도록 필요한 절차를 기술하는 것을 의미한다.

그럼 **프로그래밍 언어**(Programming Language)란 무엇일까? 언어는 사람 간에 생각을 주고 받을 수 있는 도구다. 프로그래밍 언어는 사람 대신에 컴퓨터에게 내 생각을 전달할 수 있는 도구라고 할 수 있다. 언어가 일정한 규칙으로 약속되어있듯, 프로그래밍 언어 또한 일정한 규칙이 있다. 이를 **문법**(Syntax)이라고 하며, 문법을 지키지 않으면 컴퓨터와 소통할 수 없다. 대다수의 프로그래밍 언어는 사람이 빨리 익힐 수 있게 자연어와 흡사하며, 문법 또한 대부분 비슷하다.

코드(Code)는 문법에 맞춰 적어놓은 명령어를 의미한다. 그런 의미에서 **코딩**(Coding)은 코드를 작성하는 것이라고 할 수 있다. 간혹 코딩과 프로그래밍을 동일하게 여기는 경우가 있는데, 엄밀히 다르다. 프로그래밍은 목적을 달성할 수 있게 **논리적인** 절차를 기술하는 것이며, 코딩은 이 절차를 바탕으로 명령어로 옮기는 것이다.

프로그래밍 언어는 문법을 의미하는 **코어**(Core) 부분과 문법을 이용해 많은 사람들이 편하게 프로그래밍 할 수 있도록 도와주는 **라이브러리**(Library) 부분으로 구성된다. 그래서 앞으로 우리가 배울 것은 우리의 생각을 컴퓨터에게 전달하려면 어떤 문법을 써야 하는지, 그리고 프로그래밍을 할 때 유용한 라이브러리는 무엇이고 어떻게 사용하는지를 중점으로 배우게 된다. 프로그래밍 언어는 여러 가지가 존재하지만 그 중 C++을 사용한다.

실습해보기

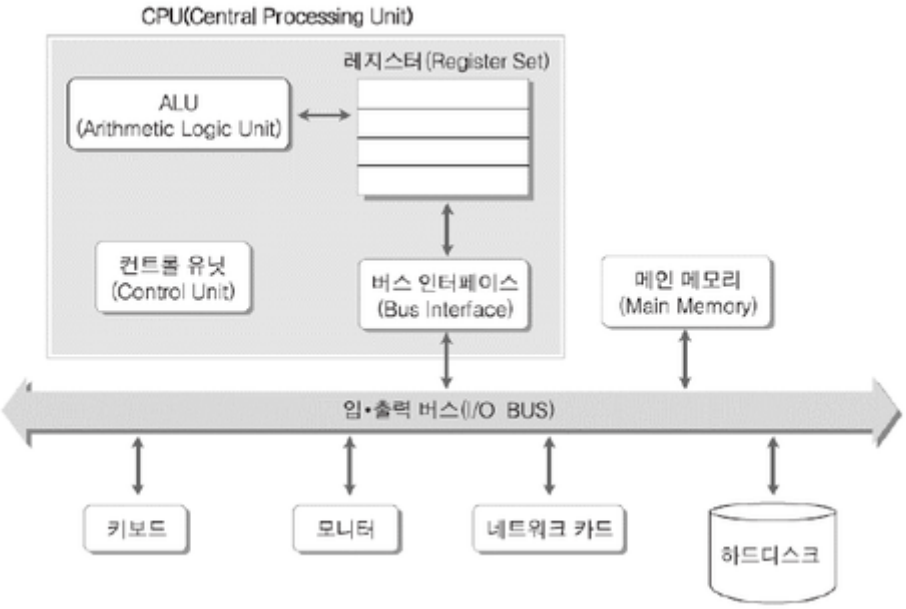
1. 프로그래밍은 무엇일까요? 프로그래밍에 대해서 정리해보세요.
2. 프로그래밍 언어란 무엇일까요? 프로그래밍 언어에 대해서 정리해보고, 프로그래밍 언어에는 무슨 언어가 있는지 찾아보세요.
3. 여러분은 이전에 코딩과 프로그래밍을 동일한 것으로 생각했나요? 그렇다면 수업을 듣고난 뒤 어떻게 생각이 바뀌었나요?

컴퓨터의 동작 원리

컴퓨터에게 우리의 생각을 알려주기 전에 컴퓨터에 대해서 먼저 이해할 필요가 있다. 간단하게 컴퓨터는 어떤 부품으로 구성되어 있고, 어떻게 동작하는지 살펴보자.

구성 요소

현대적 컴퓨터는 크게 **메모리**, **입출력 장치**, **CPU**로 나눌 수 있으며, 이 부품들이 서로 맞물려 동작한다. 각 부품은 각자만의 역할이 있다. 컴퓨터를 그림으로 나타내면 아래와 같다.



컴퓨터의 구성 요소

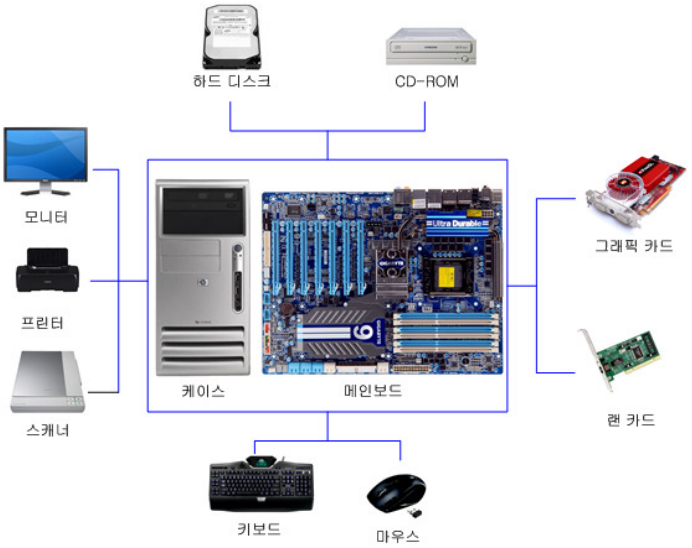
- CPU

CPU(Central Processing Unit)는 우리가 입력한 명령어의 처리를 담당하는 부품이다. 컴퓨터의 두뇌라고 할 수 있다.

~~물론 CPU가 이렇게 번쩍번쩍하게 빛나면서 동작하진 않는다.~~

- **입력과 출력**

컴퓨터는 계산하다는 의미의 Compute와 행위자를 뜻하는 er의 합성어로 본래 계산을 위해 만들어진 기계이다. 그럼 컴퓨터를 사용하기 위해 우리의 데이터를 넣어주는 방법과 그 결과를 보는 방법이 필요할 것이다. 컴퓨터로 데이터를 주는 것을 **입력**(Input)이라고 하며, 컴퓨터가 우리에게 데이터를 주는 것을 **출력**(Output)이라고 한다. 입출력 장치는 여러 가지가 있으며 우리가 그 중 흔히 사용하는 입력 장치는 키보드 및 마우스, 출력 장치는 모니터라고 할 수 있다.



입출력 장치는 다양하다.

- **비트**

컴퓨터가 사용하는 모든 데이터는 **비트**(Bit)로 저장된다. 비트는 Binary와 Digit의 합성어로, 2가지 상태를 나타낼 수 있는 수이다. 비트에 저장할 수 있는 데이터는 무엇이든지 상관 없다. 그냥 수로서 0과 1이 될 수도 있고, 화재가 발생했다 / 발생하지 않았다, 왼쪽 / 오른쪽, 존재한다 / 존재하지 않는다 등 추상적인 의미도 저장할 수 있다. 하지만 비트가 나타낼 수 있는 상태는 오직 2가지이기에 비트 하나만으로는 많은 걸 저장할 수 없다. 그래서 컴퓨터는 비트를 여러 개 묶어 **바이트**(Byte)*라는 단위로 다루게 된다.

* 1바이트는 비트 8개이다. 즉 1byte = 8bit 이다.

- **메모리**

메모리(Memory)는 비트가 저장되는 공간이다. 다시 말해 우리의 데이터가 저장되는 공간이다. 메모리는 여러 단독 주택이 모인 주택 복합 단지에 비유할 수 있는데, 각 집마다 **주소**(Address)가 있으며, 우리가 컴퓨터에 데이터를 저장할 때는 주소를 활용한다.

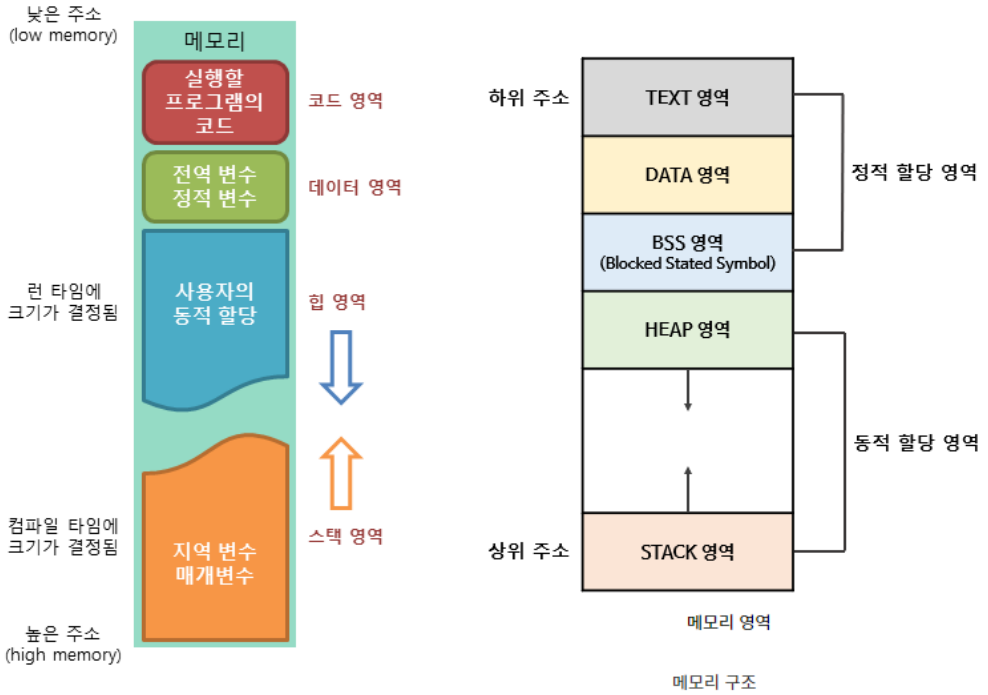
각 집에 우리의 데이터가 저장된다.

프로세스

이제까지 하드웨어에 대해서 살펴봤다. 이번엔 소프트웨어를 살펴보자. 프로그램을 실행하면 무슨 일이 일어날까? 프로그램을 실행하면 **프로세스**(Process)라는 것이 된다. 프로세스는 프로그램을 실행하기 위한 여러 가지 데이터를 관리하며 아래와 같이 4가지 영역이 존재한다.

- 코드(Code) : 명령어가 저장되는 공간이다.
- 데이터(Data) : 정적* 데이터가 저장되는 공간이다.
- 힙(Heap) : 동적* 할당 영역이다.
- 스택(Stack) : 정적 할당 영역이다.

* 컴퓨터 공학에서 정적(Static)과 동적(Dynamic)은 자주 등장하는 용어인데, 보통 정적은 프로그램 실행 전에 무언가를 한다는 것이고, 동적은 프로그램 실행 시간에 무언가를 한다는 것이다.



우리의 프로그램은 메모리를 위와 같이 사용한다.

실습해보기

1. 컴퓨터는 어떤 부품으로 구성되어 있으며, 각 부품은 무슨 역할을 하나요?
2. 비트에 담을 수 있는 데이터를 5가지 나열해보세요.
3. 프로세스가 무엇인지 정리해보세요.

프로그래밍 도구

프로그래밍을 하기 위해선 여러 가지 도구가 필요하다. 그래서 프로그래밍을 위해 어떤 도구가 있고, 그 도구의 용도는 무엇인지 명확히 알 필요가 있다. 다양한 도구가 있지만, 모두 살펴볼 순 없고 가장 기본적인 도구만 알아보도록 하자.

텍스트 에디터

텍스트 에디터는 코드를 작성하기 위해 필요하다. 여기에는 메모장, Vim, Visual Studio Code, Sublime Text 등이 있다.

컴파일러

코드를 작성했다면 이를 컴퓨터가 실제로 이해할 수 있는 명령어로 바꾸는 과정이 필요하다. 이를 컴파일(Compile)이라고 하며, 컴파일러(Compiler)는 컴파일을 하는 프로그램이다. 컴파일러는

프로그래밍 언어마다 다양하게 존재하며, 우리가 배우는 C++의 경우 GCC, Clang, MSVC 등이 있다.

디버거

프로그램을 만들다보면 우리가 모든 것에 대해 완벽하게 생각할 수 없기에 필연적으로 오류가 발생하게 된다. 이런 오류를 버그(Bug)라고 하며, 디버거(Debugger)는 버그를 고칠 수 있도록 여러 가지 기능을 제공한다.

IDE

상기된 것을 포함하여 프로그램을 개발하기 위한 여러 가지 도구를 하나의 프로그램으로 만든 것을 **통합 개발 환경**(Integrated Development Environment)이라고 한다. 종류로는 Android Studio, IntelliJ IDEA, Xcode, Visual Studio 등이 있고, 우리는 그 중 Visual Studio 라는 도구를 사용한다. 사용법에 대해서는 실습과 함께 배울 것이다.

실습해보기

1. 프로그래밍 도구는 이 외에도 여러 가지가 있습니다. 어떤 도구가 있는지 조사해보세요.
2. Visual Studio에서 제공하고 있는 기능은 무엇이 있는지 조사해보세요.

C++의 기본 개념

Hello World

C++에는 값, 객체, 레퍼런스, 함수, 열거형 등 여러 가지 구성 요소가 있다. 각 구성 요소에 대해서는 앞으로 차근차근 배워갈 것이다. 모든 프로그래밍 언어를 배울 땐 가장 기본적인 프로그램이 있는데, 바로 Hello World이다. 우선 아래의 코드를 입력해보자.

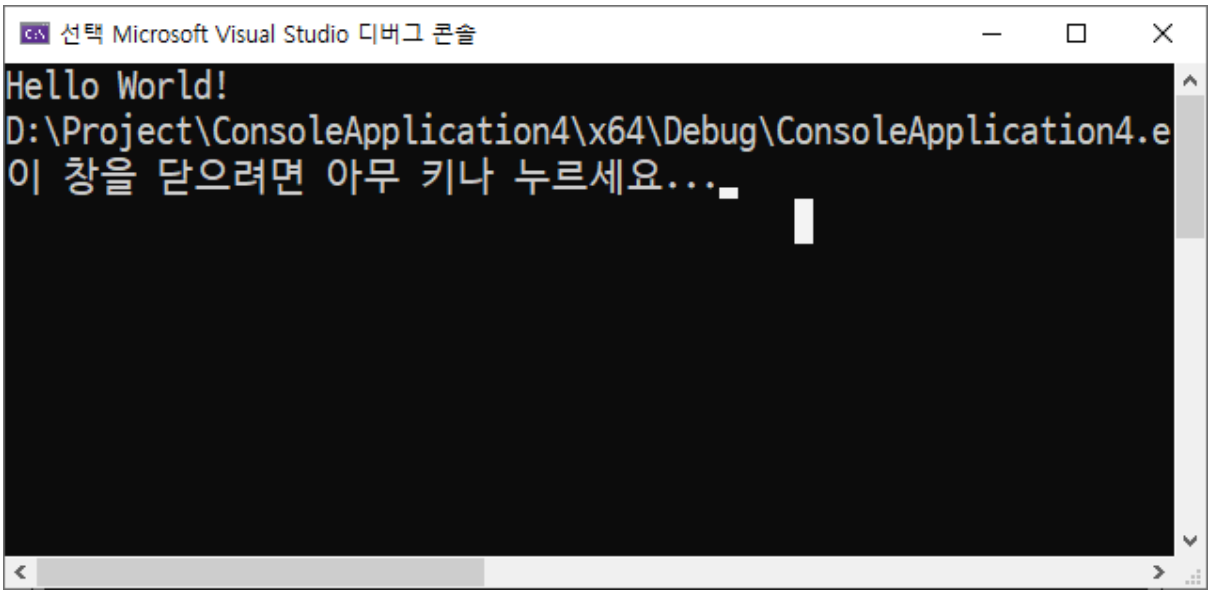
```
#include <iostream>

// 이것은 주석입니다.

int main()
{
    std::cout << "Hello World!";

    return 0;
}
```

코드를 입력하고 실행해보면 아래와 같은 창이 뜰 것이다.



이러한 창을 콘솔(Console)이라고 한다.

각각 무얼 뜻하는지 살펴보자.

```
#include <iostream>
```

<iostream>은 **헤더 파일**(Header File)로 라이브러리가 들어있는 코드 모음이라고 할 수 있다. **라이브러리**(Library)라는 것은 프로그래밍 하는 데 사용되는 여러 가지 유용한 유틸리티 모음이라고 할 수 있다. #include는 전처리기(Preprocessor)라는 것으로 이름에서 알 수 있듯 무언가를 포함한다고 할 수 있다. 정리해보면 위의 내용은 <iostream> 헤더 파일을 포함한다는 것이다.

```
// 이것은 주석입니다.
```

주석(Comment)은 프로그래머끼리 정보를 남길 수 있는 도구다. 우리가 작성하는 프로그램에는 아무 영향을 끼치지 않는다. 위처럼 // 을 쓰면 한 줄로만 주석을 작성할 수 있다. 여러 줄의 주석을 작성하고 싶다면 아래처럼 사용하면 된다.

```
/*
이렇게 쓴다면
여러 줄의 주석을 작성할 수 있습니다.
*/
```

모든 프로그램은 **시작점**(Entry Point)이 있는데, 바로 main 함수이다.

```
int main()
{

}
```

함수에 대해서는 추후 상세하게 배울 것이며, 우선은 프로그램을 만들기 위해서 저렇게 쓴다고만 알아두자. main 함수는 딱 하나만 존재해야 한다. {} 안에는 우리가 컴퓨터에게 시킬 여러 가지 명령어를 작성한다. 이번엔 main 함수의 내용물을 살펴보자.

```
int main()
{
    std::cout << "Hello World!";

    return 0;
}
```

컴퓨터에게 명령어를 내리는 것을 **구문**(Statement)이라고 하며, 구문은 세미콜론(;)으로 구분한다. 위의 main 함수에는 2가지 구문이 있는 것을 알 수 있다. 각각 살펴보자.

```
std::cout << "Hello World!";
```

위의 구문은 콘솔 창에 출력을 하기 위한 구문이다. cout은 console output이라는 의미인데 구문을 보면 마치 콘솔 창으로 데이터를 보내는 듯이 보인다. 위의 결과창에서 볼 수 있듯 Hello World! 라는 문구가 출력됨을 볼 수 있는데, 프로그래밍에선 이런 문구를 **문자열**(String)이라고 하며 큰 따옴표로 묶어서 표현한다.

```
return 0;
```

이 구문은 반환문이라고 하며, 이 구문이 가지는 의미는 차후 함수에서 다룬다.

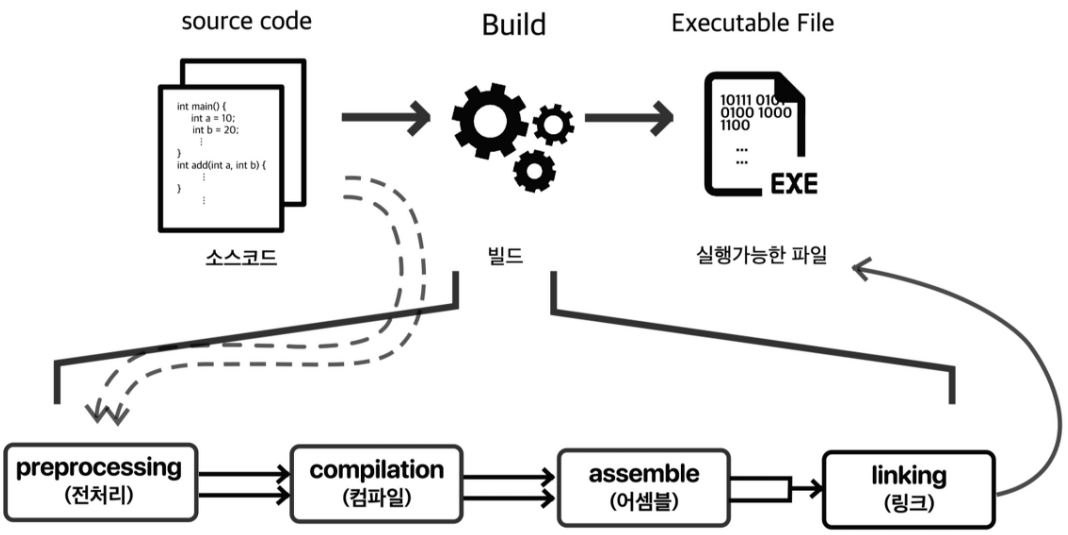
실습해보기

1. 여러분의 이름을 콘솔 창에 출력해보세요.
2. 좋아하는 영화나 책의 제목을 콘솔 창에 출력해보세요.

3. main 함수를 2개 이상 작성하고, 어떤 오류가 출력되는지 확인해보세요.

빌드

그럼 우리가 작성한 코드는 실제 어떤 과정을 거쳐서 프로그램이 되는 걸까? 코드를 바탕으로 프로그램을 만드는 과정을 **빌드**(Build)라고 한다. C++ 프로그램의 빌드 과정은 아래와 같다.



C++의 빌드 과정

- 전처리 : 컴파일 이전에 일어나는 일들을 말한다.
- 컴파일 : 우리가 작성한 코드를 어셈블리어로 변환하는 과정을 말한다.
- 어셈블 : 어셈블리어를 CPU가 이해할 수 있는 기계어로 변환하는 과정을 말한다. 어셈블 이후에는 **목적 파일**(Object File)이 생성된다.
- 링크 : 각 목적 파일을 연결해 하나로 묶어 **실행 파일**(Executable File)로 만드는 과정이다.

오류에 대한 이해

프로그래밍을 하다 보면 실수를 하고 그 실수로 인해 오류가 발생한다. 오류에는 여러 종류가 있는데, 우리는 이에 대해 이해하고 있어야 이 오류를 고치기 위해 어디서부터 출발할지 알 수 있다.

- **컴파일 오류**(Compile Error) : 가장 찾기 쉬운 오류로 문법 오류가 발생한 것이다.
- **링크 오류**(Link Error) : 링크 과정에서 발생한 오류로 어떤 이름에 대한 충분한 정보가 없을 때 발생한다.
- **런타임 오류**(Runtime Error) : 실행 시간에 발생한 오류로 버그가 발생한 것을 의미한다. 가장 찾기 어렵다.

실습해보기

4. 빌드가 무엇이고, C++ 프로그램의 빌드 과정은 어떻게 일어나는지 정리해보세요.
5. 프로그램의 오류에는 무엇이 있으며, 각 오류가 무엇이고, 각 오류에 대한 예제를 정리해보세요.

객체

입력하기

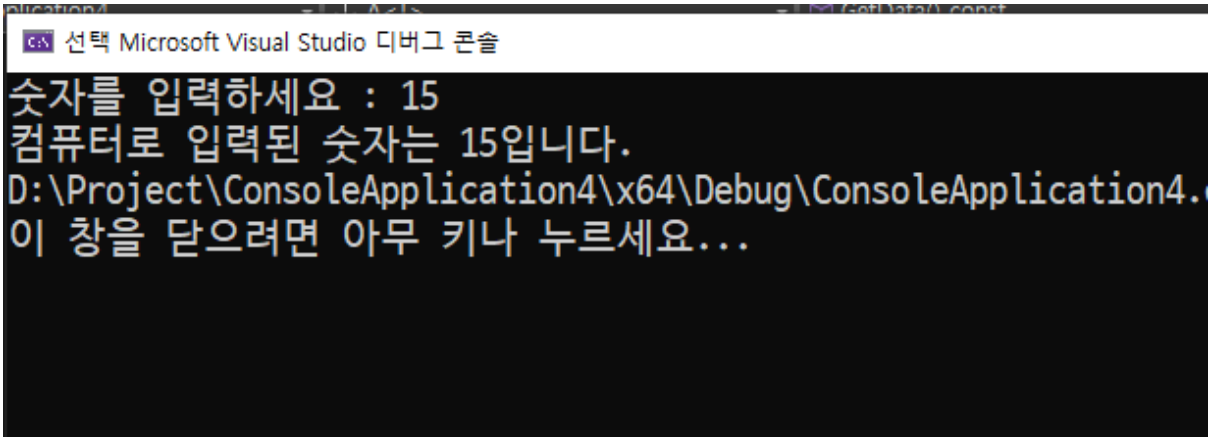
이전에 Hello, World 프로그램에서는 단순히 문자열을 출력했다. 이번에는 컴퓨터로 데이터를 보내는 법, 즉 입력하는 방법에 대해서 알아보도록 하자. 우선 아래의 코드를 입력해보자.

```
#include <iostream>

int main()
{
    std::cout << "숫자를 입력하세요 : ";
    int num;
    std::cin >> num;
    std::cout << "컴퓨터로 입력된 숫자는 " << num << "입니다.";

    return 0;
}
```

코드를 입력하면 아래와 같은 결과창이 뜬다.



우선 우리가 입력한 데이터를 저장해야 한다. 컴퓨터는 상기했듯 데이터를 저장하기 위해 메모리를 사용한다. 그럼 메모리를 어떻게 다룰 수 있을까? 메모리를 다루기 위해서는 **객체**(Object)를 사용한다.

```
int num;
```

객체를 만드는 방법은 타입과 이름을 적어주면 된다. **타입**(Type)은 데이터를 저장하기 위해 메모리를 얼마나 사용할 것인지, 그리고 어떤 데이터인지 지정하는 것이다. 타입은 프로그래밍 언어마다 종류가 다르며, C++의 경우 여기서 살펴볼 수 있다. 하지만 이중 가장 많이 사용하는 타입은 아래와 같다.

```
int integer;          // int는 정수를 나타내며, 4바이트를 사용한다.
float realNumber;     // float는 실수를 나타내며, 4바이트를 사용한다.
std::string str;      // string은 문자열을 나타낸다.
```

이름의 경우 어떤 데이터인지 잘 나타내도록 명명해야 한다.* 이름은 프로그래밍 언어에서 특수한 의미를 가지는 **키워드**(Keyword)를 제외하고 일련의 규칙에 맞춰 작성할 수 있다. (1) 숫자로 시작할 수 없으며, (2) 대소문자를 구별한다.

* 이를 위해 코딩 컨벤션(Coding Convention)이라는 것이 있다.

```
// 이름이 데이터를 나타내기에 불충분하다.
t = s + l - b;

// 이제는 이름만 봐도 어떤 데이터인지 한눈에 알 수 있다.
TotalLeaves = SmallLeaves + LargeLeaves - SmallAndLargeLeaves;
```

실습해보기

1. 객체란 무엇인가요?
2. 정수, 실수, 문자열을 입력받는 프로그램을 작성해보세요.
3. 이름을 지을 때 규칙을 지키지 않으면 어떤 일이 발생하나요?
4. 타입이란 무엇인가요?
5. 타입 중 기본 타입(Fundamental Type)이라는 것이 있습니다. C++의 기본 타입에는 무엇이 있는지 조사하고 정리해보세요.

산술 연산

이번엔 **산술 연산**(Arithmetic Operation)에 대해서 알아보자.

```
#include <iostream>

int main()
{
    std::cout << "두 개의 정수를 공백으로 구분하여 입력하세요 : ";
    int a;
    int b;
    std::cin >> a >> b;

    // 더하기(Addition)
    std::cout << "a + b = " << a + b << "\n";

    // 빼기(Subtraction)
    std::cout << "a - b = " << a - b << "\n";

    // 곱하기(Multiplication)
    std::cout << "a * b = " << a * b << "\n";

    // 나누기(Division)
    std::cout << "a / b = " << a / b << "\n";

    // 나머지(Modulo)
    std::cout << "a % b = " << a % b << "\n";

    return 0;
}
```

결과는 아래와 같다.

Microsoft Visual Studio 디버그 콘솔

```
두 개의 정수를 공백으로 구분하여 입력하세요 : 12 10
a + b = 22
a - b = 2
a * b = 120
a / b = 1
a % b = 2
```

실습해보기

1. a에 17을 b에 3을 입력했을 때 a / b의 결과는 무엇인가요?

2. a에 17을 b에 3을 입력했을 때 a % b의 결과는 무엇인가요?
3. 정수와 실수에 대한 산술 프로그램을 작성해보세요.
4. 실수와 실수에 대한 산술 프로그램을 작성해보세요.

할당 연산

꼭 입력을 받지 않더라도 객체에 데이터를 저장할 수 있다. 이를 **할당 연산**(Assignment Operation)이라고 한다.

```
#include <iostream>

int main()
{
    int num = 0; // num에 0을 저장했다.
    std::cout << num << "\n";

    num = 10 + 30; // num에 40을 저장했다.
    std::cout << num << "\n";

    int num2 = num; // 다른 메모리에 저장되어 있는 데이터를 가져올 수 있다.
    std::cout << num2 << "\n";

    // 아까 배운 산술 연산에 할당 연산을 함께 사용할 수 있다.
    num2 += 20; // num = num + 20;
    std::cout << num2 << "\n";

    return 0;
}
```

특히 할당 중 객체를 만들 때 데이터를 할당하는 것을 **초기화**(Initialization)라고 하며, 이때의 데이터를 초기값이라고 한다.

```
int num = 0; // num을 초기값 0으로 초기화했다.
```

초기화를 하지 않아 오류를 발생하는 경우도 종종 있으므로 꼭 객체를 만들 때는 초기화 하는 것을 잊지 말자.

실습해보기

1. 초기화와 할당의 차이점은 무엇인가요?

2. 초기화를 하지 않은 객체를 출력할 때, 어떤 일이 일어나는지 확인해보세요.
3. 초기화를 하지 않은 객체를 산술 연산이나 할당 연산에 활용할 때, 어떤 일이 일어나는지 확인해보세요.

증감 연산

어떤 데이터를 1 증가하거나 1 감소시키려 할 때, 아래와 같이 할당 연산을 사용할 수 있다.

```
int num = 10;
num += 1;
num -= 1;
```

이보다 좀 더 간편하게 사용할 수 있는 연산이 있는데 바로 **증감 연산**(Increment/Decrement Operation)이다.

```
int num = 10;
++num;
--num;
```

증감 연산은 전위 증감 연산과 후위 증감 연산으로 나뉘는데, 위를 전위 증감 연산이라 하며, 바로 위의 예시와 동일하다. 하지만, 후위 증감 연산은 조금 다르다.

```
int num = 10;
int num2 = num++; // num2는 10, num은 11
int num3 = num2--; // num3는 10, num2는 9
```

실습해보기

1. 아래의 연산이 끝난 후, 각 객체에 저장되어 있는 값은 무엇일까요?

```
int num = 10;
int num2 = ++num;
int num3 = num + num2--;
```

2. 실수 타입의 객체에도 증감 연산을 사용할 수 있습니다. 간단한 예제 프로그램을 작성해보세요.

비트 연산

비트 단위로도 연산을 진행할 수 있다. 비트 연산은 정수 타입에만 가능하다. 비트 연산을 이해하려면 먼저 비트로 정수를 어떻게 표현하는지 이해해야 한다. 비트로 정수를 표현하려면 이진수로 변환해야 한다. 이진수 변환 방법은 여기를 참고한다. 정수는 부호가 있는 타입과 없는 타입으로 나뉜다. 아래는 부호가 있는 타입이다.

```
signed char obj1;    // 1바이트
short obj2;          // 2바이트
int obj3;            // 4바이트
long obj4;           // 4바이트
long long obj5;      // 8바이트
```

부호를 표현하기 위해 상위 1비트*를 부호로 사용해 0이면 양수, 1이면 음수로 취급한다.

* 이를 MSB(Most Significant Bit)라고 한다.

```
signed char positive = 13;  // 0000 1101
signed char negative = -13; // 1111 0011
                            // ↑
                            // MSB
```

음수는 2의 보수를 사용해 표현한다. 2의 보수는 1의 보수에 +1을 해주면 된다. -13을 2진수로 표현해보자면 아래와 같다.

1. 먼저 13을 이진수로 표현한다. ⇒ 0000 1101
2. 1의 보수를 취해준다. ⇒ 1111 0010
3. +1을 더해준다 ⇒ 1111 0101

부호를 사용할 필요가 없다면 unsigned를 붙이면 된다.

```
unsigned char obj1;
unsigned short obj2;
unsigned int obj3;
unsigned long obj4;
unsigned long long obj5;
```

실습해보기

1. 183를 2진수로 바꿔보세요. 8비트로 표현합니다.
2. -58을 2진수로 바꿔보세요. 8비트로 표현합니다.

3. 정수에 여러 타입이 있는 것을 확인했습니다. 각 정수 타입이 저장할 수 있는 데이터의 범위를 얼마인지 조사해보세요.
4. 16진수에 대해서 조사해보고 1번과 2번을 16진수로 나타내보세요.

제어문

위에서 우리는 메모리를 어떻게 사용할 수 있는지 살펴봤다. 하지만 이것만으로는 유용한 프로그램을 작성하기가 매우 어렵다. 프로그램의 실행 흐름이 선형적이기 때문이다. 다시 말해 우리가 작성한 순서대로만 동작한다. 그래서 프로그램의 실행 흐름을 변화시킬 수 있는 구문이 있는데, 이를 제어문(Control Statement)이라고 한다. 제어문에는 크게 분기문과 반복문이 있는데 각각 살펴보도록 하자.

분기문

분기문(Branch Statement)은 실행 흐름에 분기를 만드는 구문이다. 주어진 조건에 따라 각각 다른 동작을 하도록 만들어준다. 분기문에는 if문과 switch문이 있다.

if문

아래의 코드를 작성해보자.

```
// 놀이기구 입장 가능 판단 프로그램
// 키를 cm 단위로 소수점을 제외하고 입력 받아 주어진 키가 165cm 이상이라면
"놀이기구를 탑승할 수 있습니다."를 출력한다.
#include <iostream>

int main()
{
    std::cout << "키를 입력해주세요. : ";
    int height = 0;
    std::cin >> height;

    if (height >= 165)
    {
        std::cout << "놀이기구를 탑승할 수 있습니다.";
    }

    return 0;
}
```

배열

프로그래밍을 하다 보면 비슷한 성질을 가지는 데이터를 여러 개 다뤄야 할 때가 있다. 가령 한 반에 30명인 학생의 점수를 가지고 순위를 매긴다고 해보자. 우리가 지금 할 수 있는 것은 변수를 30개를 만드는 것이다.

```
int grade1;
int grade2;
int grade3;
// ...
int grade30;
```

딱 봐도 뭔가 좋지 않다. 만일 학생 수가 30명이 아니라 100명이라면? 1000명이라면? 코드를 읽기도 매우 힘들어질 것이고, 작성하는 것도 쉽지 않을 것이다.

배열(Array)은 같은 타입으로 된 여러 개의 객체를 한 번에 다루고자 할 때 사용한다. 위의 예시를 배열로 바꿔보면 아래와 같다.

```
int grades[30];
```

데이터를 다루기가 한결 간편해졌음을 확인할 수 있다. 문법을 좀 더 자세히 살펴보자.

```
// 배열의 정의는 아래와 같다.
// type identifier[array-size];
// array-size는 정수 상수여야 한다.
// 배열의 크기는 type * array-size만큼이다.

int arr[30]; // arr는 int 타입의 객체를 30개 갖고 있는 객체다.
             // 객체 하나를 원소(Element)라 한다.
             // arr의 크기는 4 * 30 = 120 바이트다.

int arr2[5] = { 1, 2, 3, 4, 5 }; // 초기화는 {}을 사용한다.
                                 // 순서대로 각 원소에 데이터가 저장된다.
int arr3[5] = { 1, 2, 3 }; // 모든 원소에 대해 초기값을 지정하지 않을 시
                           // 자동으로 0이 된다.
int arr4[] = { 1, 2, 3 }; // 배열의 크기를 지정해주지 않고,
```

```
                                    // 초기화를 했을 때는 자동으로 크기를 계산한다.
                                    // 이 경우 arr4의 크기는 3이다.
```

메모리 상에서 배열은 각 원소가 연속적으로 배치된다. 위 예시에서 arr2를 그림으로 나타내면 다음과 같다.

데이터	1	2	3	4	5
메모리 주소	0x1234	0x1238	0x123c	0x1240	0x1244

원소에 접근하려면 [] 연산자를 사용하면 된다.

```
int arr[5] = { 1, 2, 3, 4, 5 };
arr[0]; // 배열의 첫 번째 원소. 값은 1
arr[1]; // 배열의 두 번째 원소. 값은 2
arr[2]; // 배열의 세 번째 원소. 값은 3
arr[3]; // 배열의 네 번째 원소. 값은 4
arr[4]; // 배열의 다섯 번째 원소. 값은 5
```

[] 연산자에 기입하는 숫자를 **인덱스**(Index)라고 하며, 0부터 시작함에 주의해야 한다. 이유는 후술한다.

다차원 배열

배열을 사용할 때 메모리를 논리적으로 구분하여 사용할 수 있다. 가령 학급 당 학생 수가 30명이고, 10개의 반이 있다고 해보자. 이를 배열로 나타내면 총 학생 수가 300명이므로 아래와 같이 정의할 수 있다.

```
int grades[300];
```

이 배열을 가지고 각 반의 특정 학생의 데이터에 접근하려면 약간의 계산식을 거쳐야 할 것이다.

```
int grades[300]; // 각 원소의 값은 적절하게 초기화가 되어 있다고 가정한다.
grades[2 * 30 + 2]; // 3반의 3번째 학생의 데이터에 접근했다.
```

보다시피 직관적이지 않다. 이런 경우 **다차원 배열**(Multi-dimentional Array)을 사용할 수 있다.

```
// 다차원 배열의 정의는 단순히 배열 선언자를 더 작성하면 된다.
// 늘릴 수 있는 차원 수는 메모리가 허락하는 한 무한히 늘릴 수 있다.

int arr1[5]; // 일차원 배열 (5)
int arr2[5][5]; // 2차원 배열 (5 x 5).
                // 2차원 배열은 마치 행렬처럼 생각할 수 있다.
int arr3[5][5][5]; // 3차원 배열 (5 x 5 x 5)

// 초기화는 {}을 더 써주면 된다.
int arr4[5][5] = {
    { 1, 2, 3, 4, 5 },
    { 6, 7, 8, 9, 10 }
};

int arr5[2][2][2] = {
    {
        { 1, 2 },
        { 3, 4 },
        { 5, 6 },
    },
    {
        { 1, 2 },
        { 3, 4 },
        { 5, 6 },
    },
    {
        { 1, 2 },
        { 3, 4 },
        { 5, 6 },
    }
};
```

다차원 배열을 이용해 위의 데이터를 나타내보면 아래와 같다.

```
int grades[10][30]; // 30명이 한 반이며 총 10개의 반으로 구성
grades[2][2]; // 3반의 3번째 학생의 데이터에 접근
```

실습해보기

1. 로또 번호 생성기
2. 카드 뽑기
3. 빙고

4. 달팽이 배열

함수

모듈화

문법

기본 인자 전달

함수를 선언할 때 기본 인자(Default Argument)를 전달할 수 있다.

```
void Foo(int a, int b = 10);

Foo(10, 20); // a에는 10, b에는 20이 전달된다.
Foo(10); // a에는 10, b에는 기본 인자인 10이 전달된다.
```

기본 인자는 가장 오른쪽 인자부터 전달할 수 있다.

```
void Foo(int a = 10, float b); // 컴파일 오류!
```

오버로딩

C언어에서 가장 불편했던 점이 함수의 식별자를 항상 다르게 적어줘야 한다는 것이었다. C++에서는 오버로딩(Overloadding)이라는 기능을 제공해 함수의 식별자를 꼭 다르게 적지 않아도 된다. 하지만 규칙이 하나 있는데 바로 매개변수 목록이 달라야 한다는 것이다.

```
// foo라는 동일한 식별자를 사용하고 있지만
// 매개변수 목록이 (int), (double)로 서로 다르다.
int foo(int) { return 1; }
int foo(double) { return 1; }
```

아래처럼 반환형이 다른 경우는 오버로딩이 되지 않는다.

```
int foo(int) { return 1; }
double foo(int) { return 1.0; } // 컴파일 오류!
```

오버로딩 된 함수에서 실제로 호출해야 할 함수를 바인딩하는 것은 꽤 복잡하므로 여기서는 넘어가도록 한다.

간접 참조

범위
포인터
레퍼런스

레퍼런스(Reference)는 포인터 타입처럼 간접 참조를 위한 타입으로 포인터의 2가지 불편함을 해결하기 위해 등장했다. (1) 포인터는 널(Null) 값이 포함될 수 있다. (2) 간접 참조로 데이터를 다루기 위해선 역참조 연산자를 계속 사용해야 한다.

```
void Swap(int* lhs, int* rhs)
{
    // 포인터를 안전하게 사용하려면 항상 널 체크를 해줘야 한다.
    if (lhs == NULL || rhs == NULL)
    {
        return;
    }

    // 간접 참조를 하려면 * 연산자를 사용해야 한다.
    int temp = *lhs;
    *lhs = *rhs;
    *rhs = temp;
}

void Swap(int& lhs, int& rhs)
{
    // 레퍼런스는 널 값이 들어올 수 없기 때문에 널 체크를 하지 않아도 된다.

    // 더 이상 * 연산자를 사용하지 않아도 된다.
    int temp = lhs;
    lhs = rhs;
    rhs = temp;
}

int a, b;
Swap(&a, &b); // 포인터를 사용한 버전
Swap(a, b); // 레퍼런스를 사용할 때는 & 연산자를 굳이 적지 않아도 된다.
```

당연하게도 레퍼런스에 주소 연산은 불가능하다. 그리고 레퍼런스는 꼭 초기화를 해줘야 한다.

```
int& ref; // 컴파일 오류; 초기화를 꼭 해줘야 함.
```

실습해보기

1. 이제껏 만들었던 프로그램을 전부 모듈화하여 함수로 만들어보세요.

기호 상수

const

#define

열거형

사용자 정의 타입

클래스

클래스는 사용자 정의 타입으로 아래와 같은 멤버(Member)를 가질 수 있다. 멤버는 클래스의 구성 요소를 의미한다.

- 데이터 멤버
  - 데이터를 저장한다. 다시 말해 객체다.
  - 필드(Field)라고도 한다.
- 멤버 함수
  - 타입의 기능 부분이다.
  - 메소드(Method)라고도 한다.
- 내부 타입(Nested Type)
  - 클래스 안에 다른 클래스를 만들거나, 열거형, 혹은 타입 별칭을 지정할 수 있다.
- 멤버 템플릿
  - 템플릿을 작성할 수도 있다. 템플릿은 후술한다.

클래스 타입은 아래와 같이 정의한다.

```
// class 자리에는 struct, class, union 키워드를 작성할 수 있다.
class A
{
    // 클래스 내부에서 선언된 식별자는 클래스 범위(Class Scope)를 가진다.
    int  _data; // 데이터 멤버

    void foo(); // 멤버 함수

    // 내부 클래스(Nested Class)
    class B
    {
      int _data;
    };

    // 열거형
    enum { TEMP1, TEMP2, TEMP3 };

    // 타입 재정의
    typedef B MyNestedClass;
    using MyNestedClass = B; // 바로 위 구문과 동일하다.
};

// 클래스 타입의 변수는 기본 타입과 똑같이 정의할 수 있다.
// a를 A에 대한 인스턴스(Instance)라고 한다.
A a;
```

각 멤버는 접근 한정자(Access Specifier)를 통해 클래스 외부에 공개할 것과 공개하지 않을 것을 구분지어 사용자로 하여금 설계한 의도대로만 사용할 수 있게 강제할 수 있다. 접근 한정자에는 public, protected, private이 있다. protected는 후술할 상속 파트에서 살펴보도록 한다.

```
// class의 기본 접근 지정자는 private이다.
class A
{
    // 이 아래는 private 영역이다.
    // private은 클래스 외부에서 접근할 수 없다.
    int _privateData;

    // 이 아래는 public 영역이다.
    // public은 클래스 외부에서 접근할 수 있다.
public:
```

```
    int PublicData;
};

A a;

a._privateData; // 컴파일 오류; 접근 불가능
a.PublicData; // 접근 가능
```

class 키워드 대신 struct를 사용하면 기본 접근 지정자가 달라진다.

```
// struct의 기본 접근 지정자는 public이다.
struct B
{
  int PublicData;
};
```

- **인스턴스 멤버**

클래스가 가질 수 있는 멤버는 크게 **비정적 멤버**(Non-static Member)와 **정적 멤버**(Static Member)로 나눌 수 있다. 비정적 멤버는 인스턴스가 사용하는 멤버*이며, 정적 멤버는 인스턴스와 관계없이 사용하는 멤버다.

* 그래서 비정적 멤버를 인스턴스 멤버라고도 한다.

```
class A
{
  int _data;
  double  _data2;
};

// A 타입의 인스턴스 a는 메모리에 다음과 같이 표현된다.
// [ _data(4) ][ padding(4) ][ _data2(8) ]
A a;
```

비정적 멤버 함수에서는 인스턴스의 주소를 담고 있는 this 포인터라는 것으로 인스턴스의 데이터를 접근할 수 있다.

```
class A
{
public:
```

```
  void Add(int value, double value2)
  {
    // this 포인터로 인스턴스 데이터를 접근할 수 있다.
    this->_data += value;

    // this 포인터를 생략할 수도 있다.
    _data2 += value2;
  }

private:
  int _data;
  double  _data2;
};

A a;

// 인스턴스 메소드를 호출할 때 암시적으로
// 인스턴스의 주소값이 전달되어 this 포인터의 초기값으로 사용된다.
// 즉, 인스턴스 메소드의 첫 번째 매개변수는
// 클래스 타입의 포인터라고도 할 수 있다.
// Add(int value, double value2)는
// Add(A* const this, int value, double value2)인 것이다.
a.Add(1, 1.2); // 이 예시에서는 A::Add(&a, 1, 1.2)라고도 볼 수 있다.
```

비정적 멤버 함수에는 const 수식이 가능하며, const로 수식된 비정적 멤버 함수에서는 데이터 수정이 불가능하다.

```
class A
{
public:
   // const로 수식된 인스턴스 메소드
  void Foo() const
  {
    _data += 10; // 컴파일 오류! 데이터 수정 불가능
  }

  // 오버로딩이 가능하다.
  void Foo() { _data += 10; }

  void Boo() { _data += 20; }
```

```
private:
  int _data;
};

// const로 수식 되었다면 const 메소드만 호출 가능하게 된다.
const A a;
a.Foo(); // const로 수식된 Foo()만 호출 가능
a.Boo(); // 컴파일 오류!
```

- **특수한 멤버 함수**

비정적 멤버 함수 중 컴파일러가 자동으로 만들어주는 메소드가 있다. 이를 기본 메소드(Default Method)라고 한다. 기본 메소드에는 기본 생성자(Default Constructor), 복사 생성자(Copy Constructor), 이동 생성자(Move Constructor), 소멸자(Destructor), 복사 할당 연산자(Copy Assignment Operator), 이동 할당 연산자(Move Assignment Operator)가 있다. 차례대로 살펴보자.

- **기본 생성자**

생성자는 인스턴스의 초기화를 담당한다.

```
class A
{
public:
    // 생성자의 문법은 기본적으로 반환타입을 갖지 않고,
    // 타입의 이름과 동일하게 식별자를 사용한다.
    // 매개변수가 없는 생성자를 기본 생성자라고 한다.
    A()
     : _data(0) // 이 부분을 멤버 초기화 목록(Member Initializer
List)이라고 한다.
    {
      // 초기화 외에 인스턴스를 만들면서 해야할 일들을 적는다.
    }

    // 생성자는 필요한만큼 오버로딩이 가능하다.
    A(int data) : _data(data) { }

private:
    int _data;
};
```

```
A a; // 기본 생성자가 호출된다. a._data는 0이다.
A b(10); // A(int data) 생성자가 호출된다. b._data는 10이다.
```

기본 생성자는 클래스 안에 아무런 생성자도 적어주지 않을 시 자동으로 합성된다.

```
class A
{
    // 생성자를 정의해주지 않았지만 자동으로 컴파일러에 의해 만들어진다.
    // 이 때 자동으로 만들어지는 생성자는 아래와 같다.
    // A() { }
    int _data;
};

// 멤버 중에 클래스 타입이 있는 경우 기본 생성자를 호출하게 된다.
A a;

class B
{
public:
    // 생성자를 하나 만들어주었으므로
    // 기본 생성자는 더 이상 만들어지지 않는다.
    B(int data)
     : _data(data)
    { }

private:
    int _data;
};

B b; // 컴파일 오류; 기본 생성자가 없으므로 만들 수 없음.
```

모든 생성자는 기본적으로 **변환 생성자**(Converting Constructor)라고 한다. 다시 말해 생성자에 사용한 매개변수 목록에서 클래스 타입으로 암시적 변환이 일어난다.

```
class A
{
public:
    A(int data)
     : _data(data) { }
```

```
private:
    int _data;
};

void Foo(A a);

// int 타입에서 A 타입으로의 암시적 변환이 일어남.
// void Foo(int); 함수가 있다면 의도하지 않은 결과가 일어날 수 있음.
Foo(10);
```

암시적 변환이 있어 사용이 편하기도 하지만, 때에 따라 실수로 이어질 수도 있다. 만일 이런 암시적 변환을 원치 않는다면 explicit 한정자를 사용할 수 있다.

```
class A
{
public:
    // 더 이상 암시적 변환이 불가능하며 모든 코드에서 명시적으로
    // 생성자를 호출해야 한다.
    explicit A(int data)
        : _data(data) { }

private:
    int _data;
};

void Foo(A a);

Foo(10); // 컴파일 오류! 암시적 변환 불가능.
Foo(A(10)); // 사용 가능. 명시적으로 표기해줌.
```

explicit 한정자를 사용하면 암시적 변환으로 인해 발생할 수 있는 오류를 줄일 수 있다. 즉, 좀 더 안전한 코드를 작성할 수 있다.

- **복사 생성자**

또한 생성자 중 동일 타입의 레퍼런스를 인자로 받는 생성자를 복사 생성자(Copy Constructor)라고 한다.

```
class A
{
public:
    A(int data) : _data(data) { }

    // 복사 생성자의 용도는 말 그대로 다른 인스턴스의
    // 데이터를 완전히 복사하는 것이다.
    A(const A& other) : _data(other._data) { }

private:
    int _data;
};

A a(10);
A a2(a); // 복사 생성자 호출. a2._data는 a._data와 동일하다.
A a3 = a; // 이 경우에도 복사 생성자가 호출된다.
```

복사 생성자도 기본 생성자와 마찬가지로 아무런 복사 생성자가 정의되어 있지 않을 시 자동으로 합성된다.

```
class A
{
public:
    A(int data) : _data(data) { }

    // 복사 생성자를 만들지 않았으므로
    // 컴파일 시 기본 복사 생성자가 만들어진다.
    // A(const A& other) { memcpy(this, &other, sizeof(A)); }

private:
    int _data;
};

A a = 10; // 암시적 변환.
A b = a; // 기본 복사 생성자가 호출된다.
```

특히 멤버에 포인터가 있는 경우 복사 생성자를 만들 때 주의해야 한다. 아래 예시를 보자.

```
class A
```

```
{
public:
    A() : _p(malloc(sizeof(int) * 3)) { }

    void Foo(int index, int value)
    {
      _p[index] = value;
    }
private:
    int* _p;
};

A a;
A a2 = a; // 주의! a._p와 a2._p가 같은 메모리를 가리키고 있음.

a2.Foo(2, 10); // 주의! a도 영향을 받는다.
```

a와 a2는 서로 다른 인스턴스임에도 불구하고 각자의 _p가 같은 메모리 영역을 가리키고 있어 서로 영향을 주고 있다는 것을 알 수 있다. 이를 **얕은 복사**(Shallow Copy)라고 한다. 따라서 얕은 복사를 의도한 게 아니라면 아래와 같이 바꿔줘야 한다.

```
class A
{
public:
    A() : _p(malloc(sizeof(int) * 3)) { }

    A(const A& other)
      : _p(malloc(sizeof(int) * 3))
    {
      for (int i = 0; i < 3; ++i)
      {
        _p[i] = other._p[i];
      }
    }

    void Foo(int index, int value)
    {
      _p[index] = value;
    }
private:
```

```
    int* _p;
};

A a;
A a2 = a; // 이제는 서로 다른 영역을 가리킨다.
a2.Foo(2, 10); // 이제는 a에게 영향을 주지 않는다.
```

복사 생성자에서 서로 다른 메모리 영역을 가리키되 데이터는 똑같게 만들어줬다. 이를 **깊은 복사**(Deep Copy)라고 한다.

- **소멸자**

소멸자는 객체의 수명이 다할 때 자동으로 호출되는 특수한 메소드로 자원을 정리하기 위해 사용한다.

```
class A
{
public:
    A() : _p(malloc(sizeof(int) * 3)) { }

    // 소멸자도 생성자와 마찬가지로 반환 타입이 없으며
    // 식별자는 타입 이름 앞에 ~를 붙이고
    // 아무런 매개변수도 적지 않는다.
    // 보통 이와 같이 자원을 정리할 때 사용한다.
    ~A() { free(_p); _p = nullptr; }
private:
    int*    _p;
};

{
    A a;
} // 이 부분에서 a.~A(); 가 호출된다.
```

소멸자도 정의되지 않으면 자동으로 컴파일러에 의해 합성된다.

```
class A
{
private:
    int _data;
};
```

```
{
    A a; // a의 기본 생성자가 호출된다.
} // a의 기본 소멸자가 호출된다.
```

- **복사 할당 연산자**

C++에서는 연산자 오버로딩(Operator Overloading)이 가능하다. 그 중 하나의 매개변수만 가지며 그 매개변수의 타입이 클래스 타입과 동일한 할당 연산자를 복사 할당 연산자라고 한다.

```
// 복사 할당 연산자를 구현하는 방법에는 여러가지가 있지만
// 그 중 하나인 복사 후 교환 관용구(Copy-and-Swap Idiom)에 대해서 살펴본다.
class A
{
    int _data;
public:
    // 복사 후 교환 관용구를 사용하려면 매개변수의 타입은 레퍼런스가 아니어야
한다.
    A& operator=(A a)
    {
        // std::swap()을 이용해 멤버 별로 교체해준다.
        std::swap(_data, a._data);

        return *this;
    }
};
```

이런 기본 메소드에 대해서는 default와 delete 키워드를 사용해 컴파일러가 제공하는 기본 함수를 사용하거나 삭제할 수 있다.

```
class A
{
public:
    A() = default; // 컴파일러가 제공하는 기본 생성자 사용
    ~A() = default; // 컴파일러가 제공하는 기본 소멸자 사용
    A(const A&) = delete; // 복사 생성자 삭제. A 타입의 객체는 복사 불가능.
    A& operator=(const A&) = delete; // 복사 할당 연산자 삭제. A 타입의
객체는 복사 불가능.
private:
    int _data = 10; // C++11부터 기본 멤버 초기자(Default Member
```

```
Initializer)가 제공됨
};

A a;
A b(a); // 컴파일 오류! 복사 생성자 삭제됨.
A c;
c = a; // 컴파일 오류! 복사 할당 연산자 삭제됨.
```

- **정적 멤버**

멤버를 사용할 때 static 키워드를 붙이면 정적 멤버가 된다. 정적 멤버는 프로세스 주소 공간 중 스택이나 힙이 아닌 데이터 영역을 사용하며, 타입 별로 단일의 멤버로 존재하게 된다.

```
class A
{
  static int s_data; // 정적 데이터 멤버. 이 부분은 선언이다.
  static const int CONST = 10; // 상수는 정의가 가능하다.

  int _data;
public:
  // 정적 멤버 함수
  // 인스턴스 메소드가 아니기에 const 수식이 불가능하다.
  static void Foo()
  {
    _data = 10; // 컴파일 오류! 정적 메소드기 때문에 this 포인터가 존재하지
않는다.
    Boo(); // 컴파일 오류! 마찬가지로 this 포인터가 존재하지 않는다.

    ++s_data;
  }

  void Boo()
  {
    ++_data;

    Foo(); // 인스턴스 메소드에서 정적 메소드는 호출 가능하다.
  }
};

int A::s_data = 0; // 정적 데이터 멤버 정의
```

```
A a, a2;
a.Foo(); // A::s_data == 1
a2.Foo(); // A::s_data == 2

A::Foo(); // 단독으로도 호출 가능하다.
```

실습해보기

1. 이제껏 만들었던 프로그램을 전부 클래스로 만들어보세요.

공용체

템플릿

STL

C++은 템플릿을 이용해 라이브러리를 제공하며 이를 **표준 템플릿 라이브러리**(Standard Template Library)라고 한다. 표준 템플릿 라이브러리가 제공하는 기능은 정말 많지만 이중 많이 쓰이는 것들 위주로 살펴보고자 한다.

문자열 라이브러리

C++은 문자열에 대해 세 가지로 지원하고 있다. 가장 먼저 살펴볼 것은 string이다.

string

string은 아래와 같이 사용한다.

```
#include <iostream>
#include <string> // std::string을 쓰고 싶다면 이 헤더를 포함해야 한다.
#include <assert.h>

int main()
{
    std::string s = "Hello";
    std::cout << s << "\n";

    // empty()로 빈 문자열인지 확인할 수 있다.
    if (s.empty())
    {
        std::cout << "빈 문자열이다.\n";
```

```
    }

    // length() / size()로 문자열의 길이를 알 수 있다.
    std::cout << "이 문자열의 길이는 " << s.length() << "\n";
    // std::cout << "이 문자열의 길이는 " << s.size() << "\n";

    // [] 연산자로 각 문자에 접근할 수 있다.
    // front()로 첫 번째 문자에, back()으로 마지막 문자에 접근할 수 있다.
    std::cout << "이 문자열의 3번째 문자는 " << s[2] << "\n";
    std::cout << "이 문자열의 1번째 문자는 " << s.front() << "\n";
    std::cout << "이 문자열의 마지막 문자는 " << s.back() << "\n";

    // clear()로 빈 문자열로 만들 수 있다.
    s.clear();
    if (s.empty())
    {
        std::cout << "빈 문자열이다.\n";
    }

    // push_back() / append() / + 연산자로 맨 끝에 문자(열)를(을) 삽입할 수
있다.
    s.push_back('H');    // "H"
    s.append("ello");    // "Hello"
    s += " World!";      // "Hello World!"
    std::cout << s << "\n";

    // insert()로 문자(열)를(을) 중간에 삽입할 수 있다.
    s.insert(5, " Inserted"); // "Hello Inserted World!"
    std::cout << s << "\n";

    // pop_back()으로 마지막 문자를 삭제할 수 있다.
    s.pop_back(); // "Hello Inserted World"
    std::cout << s << "\n";

    // erase()로 문자(열)를(을) 삭제할 수 있다.
    s.erase(5, 9); // "Hello World"
    std::cout << s << "\n";

    // ==, !=, <, > 연산자로 문자열을 비교할 수 있다.
    if (s == "Hello World")
    {
        std::cout << "Hello World와 같다.\n";
```

```
    }

    if (s != "Hello")
    {
        std::cout << "Hello와 다르다.\n";
    }

    if (s > "Hello")
    {
        std::cout << s << "가 Hello 뒤에 나온다.\n";
    }

    if (s < "Idle")
    {
        std::cout << s << "가 Idle 전에 나온다.\n"
    }

    // substr()로 문자열을 추출할 수 있다.
    std::cout << s.substr(6) << "\n";        // "World"
    std::cout << s.substr(0, 5) << "\n";     // "Hello"

    // 숫자와 문자열 간 변환도 가능하다.
    s = "3.14";
    float f = std::stof(s); // f(3.14)
    s = "142";
    int i = std::stoi(s); // i(142)
    s = std::to_string(f); // s("3.14")

    // 입력도 받을 수 있다.
    std::cout << "단어를 입력해주세요. : ";
    std::cin >> s;
    std::cout << "문장을 입력해주세요. : ";
    std::getline(std::cin, s);

    return 0;
}
```

string_view

읽기 전용 문자열로 널 종료 문자열만큼 다루기 쉬우면서, string만큼 풍부한 기능을 제공한다. 예제 코드는 생략한다.

널 종료 문자열

문자로 된 배열이기 때문에 아무런 기능이 없으나 대신 여러 가지 함수를 제공하고 있다. C언어의 산물이다.

입출력 라이브러리

C++에서 입출력 라이브러리는 2가지 형태로 제공된다. 하나는 객체지향적으로 만들어진 스트림 기반의 라이브러리고, 다른 하나는 C 스타일의 입출력 함수다. 또, 입출력은 파일 입출력과 문자열 입출력으로 나뉜다.

스트림 기반의 입출력

스트림 기반의 입출력 라이브러리는 기본적으로 ios_base라는 클래스를 상속 받아 구현되었기에 사용법은 모두 비슷하다. 먼저 표준 파일 입출력부터 살펴보자.

```
#include <iostream> // 표준 파일 입출력은 이 헤더를 포함해야 한다.
#include <iomanip>  // 입출력과 관련된 조작을 위한 헤더다.

int main()
{
    // 출력은 << 연산자를 사용하면 된다.
    std::cout << "Hello World!\n";

    // 포맷팅도 가능하다.
    // 1. 정수 관련 포맷
    std::cout << 12 << "\n";      // "12"
    std::cout << std::hex;        // 16진수로 출력한다.
    std::cout << 12 << "\n";      // "c"
    std::cout << std::showbase;   // 진법을 표현한다.
    std::cout << 12 << "\n";      // "0xc"
    // 다시 원래대로 되돌린다.
    std::cout << std::dec << std::noshowbase;

    // 2. 실수 관련 포맷
    std::cout << 12.34 << "\n";          // "12.34"
    std::cout << std::fixed;             // 소수점 자리를 고정시켜
표현한다.
    std::cout << 12.34 << "\n";          // "12.340000"
    std::cout << std::setprecision(12); // 소수점 자리를 12자리로
고정한다.
    std::cout << 12.34 << "\n";          // "12.340000000000"
```

```
    // 다시 원래대로 되돌린다.
    std::cout << std::defaultfloat << std::setprecision(6);

    // 3. 불리언 관련 포맷
    std::cout << std::boolalpha;    // 불리언 값을 출력할 때 숫자 대신
문자로 출력한다.
    std::cout << true << "\n";      // "true"
    // 다시 원래대로 되돌린다.
    std::cout << std::noboolalpha;

    // 4. 정렬 관련 포맷
    std::cout << std::setfill('*'); // 공백대신 *출력
    std::cout << std::setw(12);     // 출력 길이를 일시적으로 12로
지정한다.
    std::cout << std::left;         // 왼쪽 정렬
    std::cout << std::hex << std::showbase;
    std::cout << 42 << "\n";        // "0x2a********"
    // 계속 같은 길이를 사용하려면 width()을 사용한다.
    std::cout.width(12);
    std::cout << std::right; // 오른쪽 정렬
    std::cout << 42 << "\n"; // "********0x2a"
    // 다시 원래대로 되돌린다.
    std::cout << std::setfill(' ');
    std::cout.width(0);

    // 입력은 >> 연산자를 사용하면 된다.
    std::cout << "숫자를 입력하세요 : ";
    int num;
    std::cin >> num;

    // good() / fail()로 오류 여부를 알 수 있다.
    if (std::cin.good())
    {
        std::cout << "숫자를 입력함.\n";
    }
    else if (std::cin.fail())
    {
        std::cout << "숫자가 아닌 다른 값을 입력함.\n";
        std::cin.clear(); // clear()를 사용하면 객체를 정상화할 수 있다.
    }

    // bool 타입으로 변환이 가능하다.
```

```
    if (std::cin)
    {
        std::cout << "오류가 없음.\n";
    }
    else
    {
        std::cout << "오류 발생함.\n";
    }

    return 0;
}
```

다음은 파일 입출력이다.

```
#include <iostream>
#include <fstream> // 파일 입출력을 위한 헤더다.
#include <string>

int main()
{
    // 파일 출력은 ofstream을 이용한다.
    std::ofstream of("temp.txt");

    // 사용법은 표준 파일과 비슷하다.
    of << "Hello World!";

    // close()로 명시적으로 파일을 닫을 수 있다.
    // 물론 굳이 하지 않아도 소멸자에서 자동으로 호출한다.
    of.close();

    {
        // 파일 입력은 ifstream을 이용한다.
        std::ifstream ifs;

        // is_open()으로 파일을 열었는지 확인할 수 있다.
        if (false == ifs.is_open())
        {
            // open()으로 파일을 열 수 있다.
            ifs.open("temp.txt");
        }
```

```
        std::string str;
        std::getline(ifs, str);
        std::cout << str << "\n";

        // eof()로 파일의 끝에 도달했는지 알 수 있다.
        if (ifs.eof())
        {
            std::cout << "파일의 끝에 도달함.\n";
        }
    } // 자동으로 ifs.close()가 호출된다.

    // 바이너리 모드로 열고 싶다면 ios::binary를 넘겨준다.
    std::ofstream of2("temp.bin", std::ios::binary);

    return 0;
}
```

다음은 문자열 입출력이다.

```
#include <iostream>
#include <string>
#include <sstream> // 문자열 입출력을 위한 헤더다.

int main()
{
    // 문자열 입출력은 stringstream을 이용한다.
    std::stringstream ss;

    // 마찬가지로 출력은 << 연산자를 사용한다.
    ss << "Hello";

    // 입력은 >> 연산자를 사용한다.
    std::string str;
    ss >> str;
    std::cout << str << "\n";

    // str()을 이용하면 스트림에 있는 문자열을
    // std::string 객체로 변환한다.
    std::string str2 = ss.str();
    std::cout << str2 << "\n";
```

```
    return 0;
}
```

만약 사용자 정의 타입도 입출력 객체를 통해 입출력을 하고 싶다면 >> 연산자와 << 연산자를 오버로딩하면 된다.

```
#include <iostream>
#include <fstream>

class A
{
public:
    A() = default;
    explicit A(int data) : _data(data) { }

    int GetData() const { return _data; }
    void SetData(int data) { _data = data; }
private:
    int _data = 0;
};

// 출력을 재정의 하고 싶다면 아래와 비슷한 선언을 하면 된다.
std::ostream& operator<<(std::ostream& oss, const A& a)
{
    return oss << "My Data : " << a.GetData();
}

// 입력을 재정의 하고 싶다면 아래와 비슷한 선언을 하면 된다.
std::istream& operator>>(std::istream& iss, A& a)
{
    iss.ignore(sizeof("My Data : "), ':');

    int data;
    iss >> data;
    a.SetData(data);

    return iss;
}

int main()
{
```

```
    // 출력
    std::ofstream ofs("temp.txt");
    A a(14);
    ofs << a;
    ofs.close();

    // 입력
    std::ifstream ifs("temp.txt");
    A b;
    ifs >> b;
    std::cout << b;
    ifs.close();

    return 0;
}
```

C 스타일의 입출력

C언어에서는 출력엔 printf()를 입력엔 scanf()를 사용했다.

실습해보기

1. 아래의 예시와 조건을 만족하는 프로그램을 작성하시오.
    a. 이름과 나이 출력의 고정 너비는 12이다.
    b. 프로그램을 확장성이 있어야 한다. 즉, 너비를 자유롭게 조정할 수 있어야 하며, 데이터가 늘어나도 프로그램의 수정이 없어야 한다.
    c. printf()로 구현된 부분과 std::cout으로 구현된 부분이 각각 존재해야 한다.

```
Microsoft Visual Studio 디버그 콘솔
printf() 쓴 버전
---------------------------
¦Name        ¦        Age¦
---------------------------
¦이민준      ¦         31¦
¦김서준      ¦         28¦
¦최도윤      ¦         25¦
¦갈예준      ¦         28¦
---------------------------

std::cout 쓴 버전
---------------------------
¦Name        ¦        Age¦
---------------------------
¦이민준      ¦         31¦
¦김서준      ¦         28¦
¦최도윤      ¦         25¦
¦갈예준      ¦         28¦
---------------------------
```

2. 1번 프로그램을 파일에 저장하는 프로그램을 작성하시오.
3. 1번 프로그램을 문자열에 저장하여 출력하는 프로그램을 작성하시오.
4. 여러분의 타입을 만들고 입출력을 재정의해보세요.

수학 라이브러리

C++에서는 굉장히 다양한 수학 라이브러리가 있다. 무엇이 있는지는 라이브러리를 참고한다.

알고리즘 라이브러리

시간 라이브러리

객체지향 프로그래밍

C++에서는 객체지향 프로그래밍을 위한 여러가지 기능을 제공한다. 우선 객체의 청사진이 되는 클래스(Class)부터 알아보자.

클래스

상속

C++에서 상속을 하고 싶다면 클래스를 정의할 때, 뒤에 콜론을 붙이고 상속할 클래스를 적어주면 된다. 상속할 클래스를 적을 때 접근 지정자를 함께 적어줄 수 있는데, 여기서는 public에 대해서만 다룬다. 그 외의 것은 범위를 벗어나므로 여기서 다루지 않는다.*

*하지만 찾아보고 싶은 사람은 여기를 참고하라.

```
class Base
{
    // 생략
};

// 상속할 클래스를 : 뒤에 적어준다.
class Derived : public Base
{
    // 생략
};
```

상속을 받을 때는 객체의 생성과 소멸 순서가 어떻게 되는지 명확히 알고 있는 것이 중요하다.

```
class Base
{
public:
    Base() { std::cout << "Base Constructor\n"; }
    ~Base() { std::cout << "Base Destructor\n"; }
};

// 상속할 클래스를 : 뒤에 적어준다.
class Derived : public Base
{
public:
    Derived() { std::cout << "Derived Constructor\n"; }
    ~Derived() { std::cout << "Derived Destructor\n"; }
};

{
    // 생성은 부모 클래스부터 자식 클래스 순으로 하게 된다.
    Derived d;
} // 소멸은 반대로 자식 클래스에서 부모 클래스 순으로 하게 된다.
```

- **protected 접근 지정자**

protected 접근 지정자는 클래스 내부 및 자식 클래스에게 접근 권한을 부여한다.

```
class Base
{
```

```
protected:
    int _baseData;

private:
    int _privateData;
};

// 상속할 클래스를 : 뒤에 적어준다.
class Derived : public Base
{
public:
    void Foo()
    {
        _baseData = 10; // 접근 가능
        _privateData = 20; // 컴파일 오류! private은 자식 클래스에서도 접근
불가능
    }
};
```

- **public 상속의 의미**

public 상속은 부모 클래스의 모든 public 멤버를 자식 클래스의 public 멤버로 만들고, 부모 클래스의 모든 protected 멤버를 자식 클래스의 protected 멤버로 만든다.

- **다중 상속**

상속할 클래스가 꼭 한 개일 필요는 없다. 여러 개를 상속 받는 것도 가능하다.

```
class Base1 { };
class Base2 { };

// 다중 상속을 할 때는 ,로 적어주면 된다.
class Derived : public Base1, public Base2
{
    // 생략
};
```

다만 다중 상속에는 큰 문제가 있다. 아래를 보자.

```
class Base1
{
```

```
public:
    void Foo() { std::cout << "Base1의 Foo()\n"; }
};

class Base2
{
public:
    void Foo() { std::cout << "Base2의 Foo()\n"; }
};

class Derived : public Base1, public Base2
{

};

Derived d;
d.Foo(); // 컴파일 오류! Base1::Foo()와 Base2::Foo() 중 누굴 호출할 것인지
모호함.
```

이런 경우를 **죽음의 다이아몬드**(The Deadly Diamond of Death) 현상이라고 한다.

- **가상 함수**

가상 함수(Virtual Function)는 다형성을 지원하기 위한 기능이다. 가상 함수를 작성하려면 메소드 앞에 virtual 한정자를 적는다.

```
class Base
{
public:
    // 가상 함수는 앞에 virtual 키워드를 붙이면 된다.
    // Foo()는 가상 함수다.
    virtual void Foo()
    {
        std::cout << "Base::Foo()\n";
    }
};
```

가상 함수의 내용을 재정의하는 것을 **오버라이딩**(Overriding)이라고 한다.

```
class Base
```

```
{
public:
    virtual void Foo()
    {
        std::cout << "Base::Foo()\n";
    }
};

class Derived : public Base
{
public:
    // 가상 함수 재정의는 그냥 정말 재정의를 해주면 된다.
    void Foo()
    {
        std::cout << "Derived::Foo()\n";
    }
};
```

그럼 가상 함수를 작성하기만 하면 다형성을 사용할 수 있는 걸까? 그것은 아니다. 부모 클래스 타입을 가리키는 포인터나 레퍼런스로 **업캐스팅**(upcasting)*하여 다뤄야 다형성이 적용된다.

* 자식 클래스 타입에서 부모 클래스 타입으로 변환하는 것을 말한다. 반대는 **다운캐스팅**(downcasting)이다.

```
class Base
{
public:
    virtual void Foo()
    {
        std::cout << "Base::Foo()\n";
    }
};

class Derived : public Base
{
public:
    // 가상 함수 재정의는 그냥 정말 재정의를 해주면 된다.
    void Foo()
    {
        std::cout << "Derived::Foo()\n";
    }
};
```

```
Base b;
Derived d;

Base& b1 = d; // 업캐스팅
b1.Foo(); // "Derived::Foo()"
```

그럼 이런 것이 어떻게 동작하는 것일까? 우리가 일반적으로 작성하는 함수는 컴파일 시간에 어떤 함수를 호출할 것인지 결정하는 **정적 바인딩**(Static Binding)인 반면에, 가상 함수는 실행 시간에 어떤 함수를 호출할 것인지 결정하는 **동적 바인딩**(Dynamic Binding)으로 동작한다. 동적 바인딩을 위해 가상 함수의 주소가 저장되어 있는 **가상 함수 테이블**(Virtual Function Table)이 각 타입마다 존재하게 되며, 각 인스턴스마다 이를 자기 타입의 가상 함수 테이블을 가리키는 **가상 함수 포인터**(Virtual Function Pointer)를 갖고 있게 된다.

```
#include <iostream>

using namespace std;

struct Base
{
  // Base 타입의 가상 함수 테이블은 아래와 같이 구성된다.
  // vftable[0] = Base::Foo
  // vftable[1] = Base::Boo
  virtual void Foo() { cout << "Base Foo\n"; }
  virtual void Boo() { cout << "Base Boo\n"; }

  // Coo는 가상 함수가 아니기 때문에
  // 가상 함수 테이블에 들어가지 않는다.
  void Coo() { cout << "Base Coo\n"; }

// 가상 함수가 존재하기 때문에 아래와 같이
// 모든 인스턴스는 가상 함수 포인터를 갖게 된다.
// private: void** __vfptr = Base::vftable;

// 가상 함수 포인터는 인스턴스가 생성될 때 초기화 되며
// 초기값은 각 타입의 가상 함수 테이블이 된다.
};

struct Derived : Base
{
  // Derived의 가상 함수 테이블은 아래와 같이 구성된다.
```

```
  // vftable[0] = Derived::Foo
  // vftable[1] = Base::Boo
  // vftable[2] = Derived::Aoo
  void Foo() { cout << "Derived Foo\n"; }
  virtual void Aoo() { cout << "Derived Aoo\n"; }

  void Coo() { cout << "Derived Coo\n"; }
};

Base b;
Base* p = &b;
p->Foo(); // Base Foo
p->Boo(); // Base Boo
p->Coo(); // Base Coo

Derived d;
p = &d;
p->Foo(); // Derived Foo
p->Boo(); // Base Boo

// 가리키고 있는 타입이 Base이므로 Base::Coo가 호출된다.
p->Coo(); // Base Coo.

// 다운캐스팅을 하여 Derived::Coo가 호출된다.
((Derived*)p)->Coo(); // Derived Coo
```

- **추상 클래스**

추상 클래스(Abstract Class)는 인스턴스를 만들 수 없는 클래스로 상위 타입을 정의하는 데 사용한다.

```
class AbstractClass
{
  // 아래와 같은 가상 함수를 순수 가상 함수(Pure Virtual Function)라 하며
  // 보통 선언만 한다.
  virtual void Foo() = 0;
}

AbstractClass obj; // 컴파일 오류! 추상 타입은 인스턴스를 생성할 수 없다.
```

순수 가상 함수는 하위 타입으로 하여금 오버라이딩을 강제하는 효과가 있다.

```
// Abstract는 추상 클래스
struct Abstract
{
  virtual void Foo() = 0;
};

struct Base : Abstract
{
  // 순수 가상 함수를 오버라이딩 하지 않으면
  // Abstract::Foo()를 그대로 Base가 갖게 되며,
  // Base 또한 추상 클래스가 된다.
};

// Derived는 Foo()를 오버라이딩 하였으므로
// 추상 클래스가 아닌 구체 클래스(Concrete Class)다.
struct Derived : Base
{
  void Foo() { }
};

Abstract temp; // 컴파일 오류! Abstract는 추상 타입
Base temp; // 컴파일 오류! Base는 추상 타입
Derived d; // OK. Derived는 구체 타입
```

- **동적 할당**

객체지향 프로그래밍을 지원하면서 동적 할당의 방법도 바뀌게 되었다. 왜냐하면 클래스 타입의 객체를 동적 할당할 때, 생성자와 소멸자를 호출해야 하기 때문이다.

```
struct A
{
  A() { std::cout << "Constructor"; }
  ~A() { std::cout << "Desturctor"; }
};

int main()
{
  int* p = new int; // 할당에는 new 연산자를
  delete p; // 해제에는 delete 연산자를 사용한다.
```

```
  p = new int[3]; // 배열의 할당이 필요할 때는 new[] 연산자를 사용하며
  delete[] p; // 해제에는 delete[] 연산자를 사용한다.

  // 가장 크게 달라지는 부분은 클래스 타입을 동적 할당 할 때이다.
  // new / delete 연산자는 클래스 타입일 때
  // 자동으로 생성자와 소멸자를 호출한다.
  A* a = new A();
  delete a;

  return 0;
}
```

new / delete 연산자의 쌍은 꼭 맞춰줘야 하는데, 동작 방식이 서로 다르기 때문이다.

- **가상 소멸자**

동적 할당을 알게 됨으로써 문제점이 있다. 아래를 보자.

```
struct Base
{
  Base() { std::cout << "Constructor"; }
  ~Base() { std::cout << "Destructor"; }
};

struct Derived : Base
{
  Derived() { std::cout << "Constructor"; }
  ~Derived() { std::cout << "Destructor"; }
};

{
  Derived d;
} // 정상적으로 생성과 소멸이 이뤄짐.
```

위와 같은 경우에는 객체의 생성과 소멸이 잘 이뤄진다. 하지만 동적할당일 땐 어떨까?

```
Base* b = new Derived();
delete b; // ~Base()만 호출됨.
```

보통 객체지향에선 업캐스팅하여 객체를 다루게 되는데, 이 경우 객체의 소멸이 제대로 안되는 것을 확인할 수 있다. 왜냐하면 결국 상위 타입으로 다루고 있기 때문이다. 따라서 올바르게 소멸자를 호출하고 싶다면 가상 소멸자(Virtual Destructor)를 정의해야 한다.

```
struct Base
{
  Base() { std::cout << "Constructor"; }
  virtual ~Base() { std::cout << "Destructor"; }
};

struct Derived : Base
{
  Derived() { std::cout << "Constructor"; }
  ~Derived() { std::cout << "Destructor"; }
};

Base* b = new Derived();
delete b; // 이제는 올바르게 소멸된다.
```

따라서 상위 타입이 될 여지가 있는 클래스라면 꼭 가상 소멸자를 정의해주도록 하자.

일반화 프로그래밍

일반화 프로그래밍(Generic Programming)은 타입에 관계없이 알고리즘을 기술하는 프로그래밍 패러다임이다. C++에서는 이를 템플릿(Template)이라는 기능으로 지원하고 있다. 이 파트에서는 템플릿에 대해 알아보고자 한다.

템플릿 문법

템플릿은 기본적으로 클래스 템플릿(Class Template)과 함수 템플릿(Function Template)으로 구분된다.*

* C++14부터는 별칭 템플릿(Alias Template), 변수 템플릿(Variable Template), 컨셉(Concept) 등을 만들 수 있다.

```
// 문법은 아래와 같이 구성된다.
// template <parameter-list> declaration

// 클래스 템플릿을 정의했다.
template <typename T>
struct A { T Data; };
```

```
// 함수 템플릿을 정의했다.
template <typename T>
void Swap(T& lhs, T& rhs)
{
    T temp = lhs;
    lhs = rhs;
    rhs = lhs;
}

// 템플릿을 사용하려면 매개변수 목록에 인자를 전달하면 된다.
// 이를 특정 타입(Specific Type)으로 대체한다고 하여
// 특수화(Specialization)라고 한다.
A<int> a; // a는 int 타입의 Data를 가진다.
A<double> b; // b는 double 타입의 Data를 가진다.

int a = 10, b = 20;
Swap<int, int>(a, b);
Swap(a, b); // 함수 템플릿은 <>를 생략할 수 있다.
```

템플릿은 하나 이상의 **템플릿 파라미터**(Template Parameter)와 함께 매개변수화 된다. 템플릿 파라미터에는 **비 타입 파라미터**(Non-type Parameter), **타입 파라미터**(Type Parameter), **템플릿 파라미터**(Template Parameter)가 있다. 정리하자면 타입도 마치 인자처럼 전달할 수 있다는 것이다.

```
// 타입 파라미터는 타입을 받을 수 있다.
template <typename T> class A { };
template <class T> class B { }; // typename 대신 class를 사용할 수 있다.
template <typename T = int> class C { }; // 기본 인자 전달도 가능하다.
// C<> c; c는 C<int> 타입이다.

// 비 타입 파라미터는 구체적인 타입을 지정하는 것이다.
// 비 타입 파라미터에 넣을 수 있는 타입으로는
// 정수형, 포인터, 레퍼런스 등이 있다.
// 마찬가지로 기본 인자 전달이 가능하다.
template <size_t N = 10> struct Array { int Container[N]; };

// 템플릿 파라미터는 템플릿을 전달하는 것이다.
template <typename T> class Container { };
template <template<typename> typename Container = Container<int>>
class Temp { };
```

특수화

특수화는 컴파일 타임에 일어나며, 컴파일러가 템플릿을 기반으로 특정 타입을 생성한다.

```
template <typename T, size_t N>
struct IdiotArray { T Container[N]; };

IdiotArray<int, 10> arr;
// 컴파일러는 컴파일 타임에 아래와 같은 코드를 생성한다.
// template <>
// struct IdiotArray { int Container[10]; };
```

따라서 일반적으로 템플릿은 헤더 파일에 모든 내용을 적으며, 정의를 소스 파일에 나눠서 적으면 링크 오류가 발생한다.

```
// Temp.h
template <typename T>
class A
{
public:
    void SetData(const T& data);
    T GetData() const;
private:
    T _data;
};

// Temp.cpp
// 이 파일 아래에 있는 템플릿은 인스턴스가 만들어지지 않는다.
template <typename T>
void A<T>::SetData(const T& data)
{
    _data = data;
}

template <typename T>
T A<T>::GetData() const
{
    return _data;
}
```

```
// Main.cpp
int main()
{
    A<int> a;
    // 아래와 같이 인스턴스가 만들어진다.
    // 정의가 없음을 볼 수 있다.
    // template <>
    // class A
    // {
    // public:
    //     void SetData(const int& data);
    //     int GetData() const;
    // private:
    //     int _data;
    // };

    a.SetData(10); // 링크 오류! 정의가 없음.
}
```

사용자가 특정 템플릿 인자에 대해 코드를 커스터마이징 할 수도 있는데, 모든 템플릿 파라미터에 대해 특수화를 하는 걸 **명시적 특수화**(Explicit Specialization), 일부만 하는 걸 **보분 특수화**(Partial Specialization)라고 한다.

```
template <typename T1, typename T2>
void Print(const T1& a, const T2& b)
{
    std::cout << a << '\n';
    std::cout << b << '\n';
}

// 명시적 특수화
template <>
void Print(const int& a, const double& b)
{
    std::cout << "이것은 명시적으로 특수화 되었습니다.\n";
}

// 부분 특수화
template <typename T>
void Print(const T& a, const int& b)
```

```
{
    std::cout << "이것은 부분적으로 특수화 되었습니다.\n";
}

double d = 1.0;
Print(d, d); // Print<double, double>; 암시적 특수화 버전 호출

int i = 1;
Print(i, d); // Print<int, double>; 명시적 특수화 버전 호출

Print(d, i); // Print<double, int>; 부분 특수화 버전 호출
```

STL

STL의 사용법에 대해서 알아보자

<vector>

```
#include <iostream>
#include <vector>

using namespace std;

int main()
{
    // 벡터 생성하기
    vector<int> vec;
    cout << boolalpha << vec.empty() << endl;
    cout << vec.size() << endl;
    cout << vec.capacity() << endl;

    // 삽입
    // push_back : 벡터의 맨 끝에 데이터 삽입
    vec.push_back(3);
    vec.push_back(4);
    vec.push_back(2);
    // { 3, 4, 2 }

    // 반복자
    // begin() : 컨테이너의 첫 원소를 가리키는 반복자
    // end() : 컨테이너의 마지막 원소의 다음을 가리키는 반복자
```

```
    for (vector<int>::iterator iter = vec.begin(); iter != vec.end();
++iter)
    {
        cout << *iter << ",";
    }
    cout << endl;

    // rbegin() : 컨테이너의 마지막 원소를 가리키는 반복자
    // rend() : 컨테이너의 첫 원소의 이전을 가리키는 반복자
    for (vector<int>::reverse_iterator iter = vec.rbegin(); iter !=
vec.rend(); ++iter)
    {
        cout << *iter << ",";
    }
    cout << endl;

    // insert : pos 이전에 데이터를 삽입
    vector<int>::iterator iter = vec.begin();
    ++iter; // 2번째 원소 : 4

    iter = vec.insert(iter, 10); // vec : { 3, 10, 4, 2 }, iter :
&vec[1] (10)
    iter = vec.insert(iter, 3, 5); // vec : { 3, 5, 5, 5, 10, 4, 2 },
iter : &vec[1] (5)
    iter = vec.insert(iter, vec.begin() + 3, vec.end() - 1); // vec :
{ 3, 5, 10, 4, 5, 5, 5, 10, 4, 2 }
                                        // iter : &vec[1] (5)
    for (vector<int>::iterator iter = vec.begin(); iter != vec.end();
++iter)
    {
        cout << *iter << ",";
    }
    cout << endl;

    // 삭제
    // pop_back : 맨 끝에 있는 데이터를 삭제
    // vec : { 3, 5, 10, 4, 5, 5, 5, 10, 4, 2 }
    vec.pop_back();
    vec.pop_back();
    vec.pop_back();
    // vec : { 3, 5, 10, 4, 5, 5, 5 }
```

```
    // erase : pos에 있는 데이터를 삭제
    vec.erase(vec.begin());
    // vec : { 5, 10, 4, 5, 5, 5 }
    vec.erase(vec.begin() + 1, vec.begin() + 3);
    // vec : { 5, 5, 5, 5 }

    // 읽기
    cout << vec.front() << endl; // 5
    cout << vec.back() << endl; // 5
    cout << vec[2] << endl; // 5

    // 검색 => 메소드가 아니라 알고리즘 라이브러리에 존재

    // 다른 생성
    vector<int> vec2(5); // { 0, 0, 0, 0, 0 }
    vector<int> vec3(5, 10); // { 10, 10, 10, 10, 10 }

    return 0;
}
```

참고자료

- 한 권으로 읽는 컴퓨터 구조와 프로그래밍 - YES24
- cppreference.com
- 5 일반화 프로그래밍(Generic Programming) - C++ Template Note
- Generic programming - Wikipedia
- 프로그래밍 언어의 단어와 문법 - 1 [프로그래밍]